IoT Secure Gateway: Network-Based Defense Against IoT Botnets Phase 3: Docker-Based Implementation and Evaluation

Dylan Luong
Algoma University
Student ID: 5150921
Brampton, Canada

Jehosua Alan Joya Venegas
Algoma University
Student ID: 5149910
Brampton, Canada

Saurodeep Majumdar
Algoma University
Student ID: 5153010
Brampton, Canada

***Abstract*—This paper presents a Docker-based implementation and evaluation of an IoT Secure Gateway designed to mitigate Mirai-style IoT botnet threats through network-layer controls. The experimental platform consists of three isolated roles: a firewall/gateway container, an internal IoT container, and an external WAN container that serves both as attacker and controlled service endpoint. The proposed architecture enforces network segmentation, stateful forwarding control, default-deny filtering, and least-privilege outbound policy using `nftables`. The gateway blocks unsolicited WAN-to-IoT SSH and Telnet-style traffic, prevents insecure outbound SSH/Telnet propagation attempts from the IoT segment, and preserves legitimate outbound HTTPS communication. A phase 3 evaluation is conducted with automated evidence collection to assess three dimensions of system performance: accuracy, efficiency, and scalability. The results show filtered inbound ports, correct firewall counter progression, low-latency legitimate HTTPS connectivity, and stable policy enforcement under burst attack conditions. These findings indicate that gateway-level filtering provides a practical and effective mitigation layer for unmanaged or legacy IoT devices.**

***Index Terms*—IoT Security, Mirai, Botnets, Docker, Docker Compose, Firewall, nftables, Network Segmentation, Stateful Filtering.**

I. Introduction

THE rapid growth of Internet-connected consumer and edge devices has expanded the attack surface available to botnet operators and other adversaries. This risk became especially visible in the Mirai botnet, which scanned the Internet for devices exposing management services such as Telnet and SSH, then enrolled compromised devices into a command-and-control infrastructure used for large-scale DDoS attacks [1]. Because many IoT devices provide limited patching support, weak default security, and little or no host-based protection, network-layer defenses remain an important mitigation strategy.

This paper presents an IoT Secure Gateway that applies segmentation and firewall enforcement to reduce exposure of a protected IoT device to Mirai-style attack behavior. The design places the IoT device behind a dedicated gateway that serves as the only routing boundary between the internal subnet and an external WAN segment. The gateway enforces a default-deny forwarding policy, blocks administrative services associated with Mirai-style scanning and propagation, and preserves only explicitly permitted outbound communication.

The paper contributes the following:

- **A reproducible Docker-based security testbed** composed of a firewall/gateway container, a protected IoT container, and an external WAN container, thereby preserving explicit WAN–LAN separation and centralized policy enforcement.
- **A case-study-driven enforcement model** that maps Mirai's documented infection workflow to concrete network controls, including default-deny forwarding, inbound blocking, and least-privilege outbound filtering [1].
- **An evidence-based phase 3 evaluation** that measures system accuracy, efficiency, and scalability through automated attack execution, firewall counter analysis, connection-timing measurements, and burst-load testing.

The implementation code, Docker configuration, automation scripts, and supporting project files used in this phase are available in the public repository [Online]. Available: https://github.com/Jehosua97/iot-secure-gateway-mirai-mitigation.

The remainder of this paper is organized as follows. Section II presents the case-study-driven design rationale based on Mirai. Section III describes the network architecture and topology. Section IV details the container configuration and routing model. Section V explains the firewall enforcement model. Section VI presents the functional attack simulation and prototype validation. Section VII introduces the phase 3 experimental methodology and results. Section VIII discusses real-world impact and identified gaps. Section IX concludes with key findings and future work.

II. Case Study–Driven Design Rationale

Mirai succeeded because many IoT devices were exposed with weakly protected administrative services, especially Telnet, and could initiate unrestricted outbound communication after compromise [1]. Once infected, devices were not only victims but also participants in botnet propagation and DDoS activity.

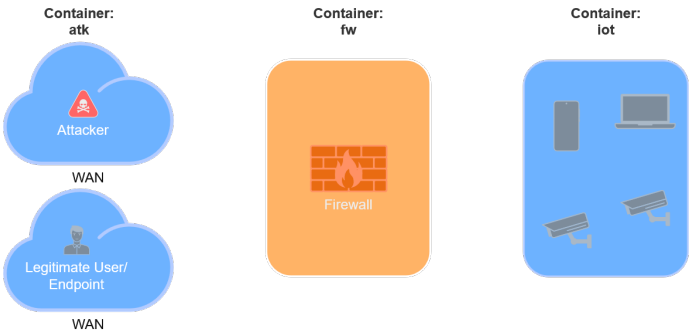

Fig. 1. High-level Docker-based network architecture for the IoT Secure Gateway lab.

Key weaknesses associated with Mirai-style behavior include:

- Exposed SSH or Telnet-like services
- Poor segmentation between untrusted and internal devices
- Unrestricted outbound communication from compromised IoT systems
- Limited patching or endpoint security on the IoT device itself

Industry guidance from Cloudflare and CISA emphasizes that segmentation, inbound filtering, and restriction of unnecessary traffic are practical ways to reduce IoT botnet risk [2], [3]. These observations motivate a network-based defense model in which the gateway, rather than the endpoint, becomes the primary enforcement point.

Accordingly, the proposed enforcement policy is designed to:

- block unsolicited WAN-to-IoT access on ports `22`, `23`, and `2323`;
- block insecure IoT-to-WAN SSH/Telnet-style traffic on ports `22`, `23`, and `2323`;
- permit only the legitimate outbound service used in the evaluation, namely HTTPS on port `443`; and
- centralize all policy decisions at the gateway firewall.

III. System Architecture

A. Overview of the Topology

The experimental platform consists of three Docker containers:

- **fw** – Firewall / Gateway
- **iot** – Simulated IoT device
- **atk** – External WAN host used both as attacker and as the controlled WAN-side service endpoint

Two Docker bridge networks provide logical separation between the protected IoT segment and the external WAN segment. The firewall container is the only multi-homed node and therefore serves as the sole routing boundary between the two networks. This structure intentionally mirrors a practical gateway deployment in which externally sourced traffic and internally initiated traffic must both traverse the same policy enforcement point.

TABLE I
Docker Container Addressing and Role Mapping

Container	Network Attachment	IP Address	Role
`fw`	`iot_net`, `wan_net`	10.20.0.1, 192.168.56.2	Router / Firewall
`iot`	`iot_net`	10.20.0.10	Protected IoT endpoint
`atk`	`wan_net`	192.168.56.10	WAN host / attacker / test service

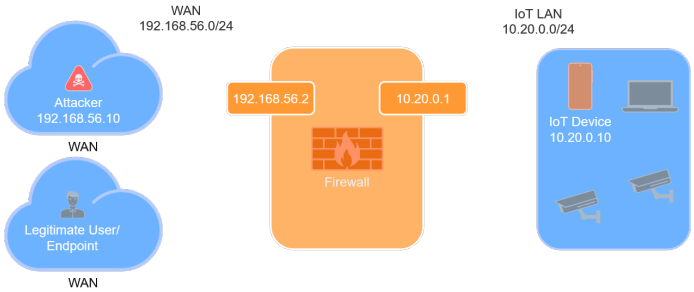

Fig. 2. Detailed Docker interface mapping and addressing for the lab environment.

B. Addressing Plan

The addressing model is shown in Table I.

This arrangement preserves the intended trust boundary:

- the IoT device is not directly reachable from the WAN without passing through the firewall
- the external host reaches the IoT subnet only through the firewall
- all meaningful policy decisions occur in the firewall container

IV. Container Configuration and Routing

A. Firewall Container

The firewall container is multi-homed. At startup, it identifies its IoT-side and WAN-side interfaces by IP address and generates the `nftables` ruleset dynamically. Layer-3 forwarding is enabled through:

```
net.ipv4.ip_forward = 1
```

To support reproducibility and later analysis, the firewall stores startup evidence under `docker-lab/results/`, including interface state, routing tables, and the active ruleset.

B. IoT Container

The IoT container has a single interface on the `10.20.0.0/24` network and installs the firewall as its default gateway:

```
default via 10.20.0.1
10.20.0.0/24 dev eth0 proto kernel scope link
```

The container also starts controlled listeners on TCP `22`, `23`, and `2323`. These services are intentionally exposed so that scan results reporting `filtered` can be attributed to firewall enforcement rather than to the absence of a listening application.

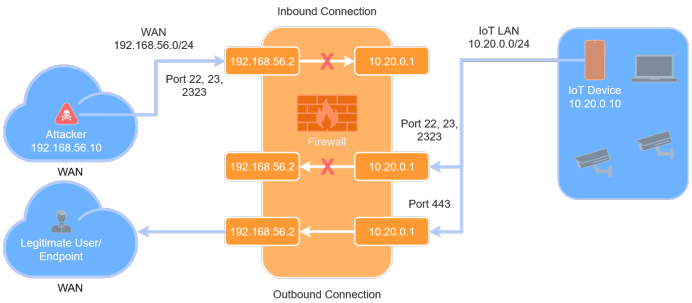

Fig. 3. Forwarding policy behavior between the WAN and IoT segments.

C. Attacker Container

The attacker container has a single interface on `192.168.56.0/24` and adds a route toward the IoT subnet through the firewall:

```
10.20.0.0/24 via 192.168.56.2 dev eth0
```

It also exposes controlled WAN-side listeners on TCP `22`, `23`, `2323`, and `443`. The `443` listener provides a controlled WAN-side destination for legitimate outbound testing, while the other ports are used to represent management services that should be blocked by policy.

V. Firewall Enforcement Model

The firewall policy is implemented in the `forward` chain of `nftables` and follows a default-deny model:

```
policy drop
```

The forwarding rules can be summarized as follows:

1) Allow established and related forwarded traffic
2) Drop WAN-to-IoT traffic targeting TCP `22`, `23`, and `2323`
3) Drop any other forwarded WAN-to-IoT traffic
4) Drop IoT-to-WAN traffic targeting TCP `22`, `23`, and `2323`
5) Allow IoT-to-WAN traffic targeting TCP `443`

Conceptually, the policy behaves as:

```
WAN -> IoT : 22,23,2323 => DROP
WAN -> IoT : everything else => DROP
IoT -> WAN : 22,23,2323 => DROP
IoT -> WAN : 443 => ACCEPT
return traffic for accepted flows => ACCEPT
```

This policy directly reflects the Mirai threat model. The botnet relied on exposed management services for initial infection and unrestricted communication for later coordination or propagation [1]. The gateway addresses both behaviors at the network boundary.

VI. Functional Validation

A. Starting the Lab

The experimental environment is launched with:

```
docker compose up --build -d
docker compose ps
```

These commands start the three-container topology and confirm that the environment is operational.

B. Inbound Mirai-Style Scanning

Inbound Mirai-style scanning is emulated from the WAN-side attacker container:

```
docker compose exec atk nmap -Pn -p 22,23,2323
    10.20.0.10
docker compose exec atk nc -nvz -w 5 10.20.0.10 22
docker compose exec atk nc -nvz -w 5 10.20.0.10 23
docker compose exec atk nc -nvz -w 5 10.20.0.10 2323
```

The observed result was:

```
22/tcp filtered ssh
23/tcp filtered telnet
2323/tcp filtered 3d-nfsd
```

This result indicates that the firewall prevents the WAN host from establishing SSH or Telnet-style sessions with the protected IoT device.

C. Outbound Policy Validation

Blocked outbound connections from the IoT container are tested with:

```
docker compose exec iot nc -nvz -w 5 192.168.56.10
    22
docker compose exec iot nc -nvz -w 5 192.168.56.10
    23
docker compose exec iot nc -nvz -w 5 192.168.56.10
    2323
```

The allowed outbound service is tested with:

```
docker compose exec iot nc -nvz -w 5 192.168.56.10
    443
```

Observed behavior:

- outbound `22`, `23`, and `2323` timed out and were blocked
- outbound `443` completed successfully

These outcomes demonstrate the core functional claim of the system: the gateway enforces a meaningful separation between hostile or unnecessary traffic and legitimate communication.

VII. Phase 3 Experimental Methodology and Results

The phase 3 evaluation is organized around three performance dimensions:

- accuracy
- efficiency
- scalability

Evidence was collected automatically with:

```
powershell -ExecutionPolicy Bypass -File .\run-
    phase3.ps1
```

The resulting run created a timestamped evidence folder:

```
docker-lab/results/20260401-123110
```

A. Accuracy

Accuracy evaluates whether the firewall applies the intended policy correctly.

Three observations support the accuracy claim:

1) the inbound scan output showed `filtered` results for `22`, `23`, and `2323`
2) the firewall counter after inbound testing showed:

```
block wan ssh telnet to iot -> 15 packets, 804
    bytes
```

3) the outbound policy counters after blocked and allowed tests showed:

```
block iot ssh telnet outbound -> 9 packets, 540
     bytes
allow iot https to wan -> 5 packets, 300 bytes
```

Taken together, these results support policy accuracy:

- unsolicited WAN-to-IoT management traffic was blocked
- insecure IoT-to-WAN management traffic was blocked
- legitimate IoT-to-WAN HTTPS traffic was allowed

B. Efficiency

Efficiency evaluates whether the security policy preserves legitimate functionality with low overhead.

The automated run measured repeated outbound `443` timing before the scalability burst:

```
run=1 status=success connect_ms=3
run=2 status=success connect_ms=3
run=3 status=success connect_ms=3
run=4 status=success connect_ms=3
run=5 status=success connect_ms=2
```

After the scalability burst, HTTPS remained available with similarly low timing:

```
run=1 status=success connect_ms=2
run=2 status=success connect_ms=2
run=3 status=success connect_ms=2
run=4 status=success connect_ms=2
run=5 status=success connect_ms=1
```

Resource snapshots also remained low. Baseline container memory usage was approximately:

- firewall: `3.035 MiB`
- IoT device: `3.219 MiB`
- attacker: `4.035 MiB`

After scaling, container memory remained in the same small range and firewall CPU usage stayed negligible in the sampled output. This suggests that the gateway preserved legitimate WAN-side HTTPS connectivity while imposing very low visible overhead in the laboratory environment.

C. Scalability

Scalability evaluates whether the firewall continues to enforce policy as attack intensity increases.

The automated script executed burst tests with 10, 25, and 50 parallel blocked connection attempts:

```
parallel_connections=10 elapsed_seconds=3
parallel_connections=25 elapsed_seconds=3
parallel_connections=50 elapsed_seconds=3
```

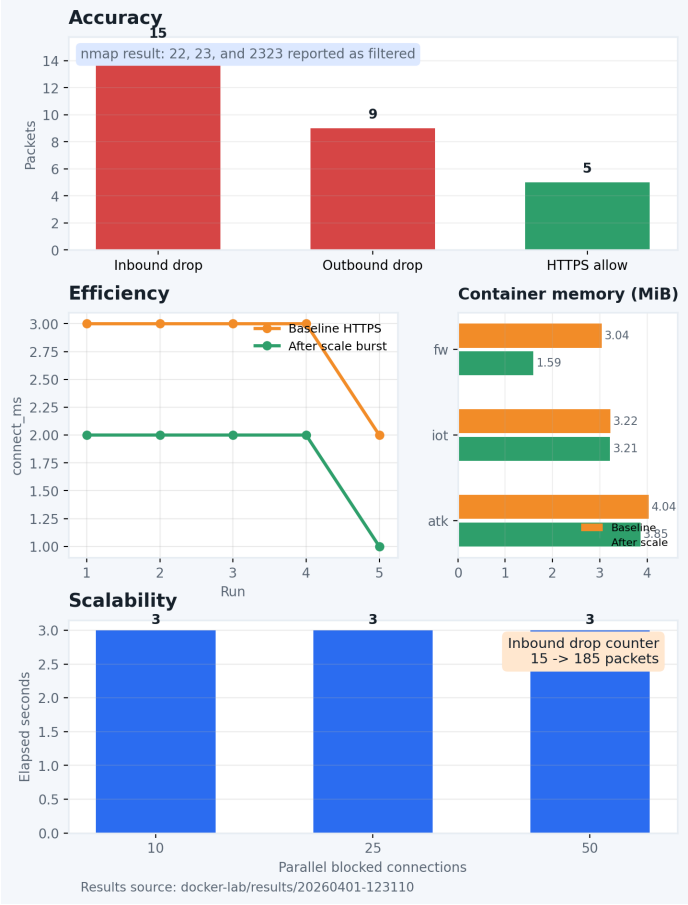

Fig. 4. Phase 3 performance summary showing policy accuracy, low-overhead HTTPS connectivity, and stable behavior under increasing blocked-connection bursts.

After those bursts, the cumulative inbound drop counter increased from:

```
15 packets after inbound accuracy testing
```

to:

```
185 packets, 11004 bytes after scalability testing
```

Most importantly, the legitimate `443` test remained successful after the burst. This shows that the firewall continued to enforce the blocking policy while preserving the allowed service path.

D. Interpretation of Phase 3 Results

Taken together, the phase 3 results provide strong support for the system's performance claims:

- **Accuracy** was demonstrated through exact rule matches and expected `filtered` or timeout behavior
- **Efficiency** was supported by repeated successful HTTPS timings and low container resource usage
- **Scalability** was demonstrated by increasing burst size while maintaining correct policy enforcement and preserving legitimate traffic

Because the evidence is timestamped, repeatable, and generated automatically, it provides a stronger basis for evaluation than a one-off manual demonstration.

VIII. Real-World Impact and Identified Gaps

A. Real-World Impact

The project has a strong connection to real-world IoT risk for several reasons.

First, many consumer and edge IoT devices still expose weak management paths, are not patched frequently, and cannot easily run endpoint security tools. A network-based gateway is therefore practical because it protects the device without requiring firmware changes.

Second, the policy directly addresses a known real-world infection model. Mirai did not rely on highly sophisticated zero-day techniques. It exploited exposed services, weak credentials, and broad Internet reachability [1]. Blocking unnecessary WAN-to-IoT access and constraining risky outbound communication is therefore a realistic and meaningful mitigation strategy.

Third, this architecture is deployable beyond a lab. Home routers, small-office gateways, and edge appliances can implement the same security principle: place IoT devices in a separate segment, filter inbound traffic by default, and allow only the minimal outbound services required for operation.

B. Identified Gaps

The current implementation is effective, but it is not a complete defense.

Key limitations include:

- the firewall permits `443` based on port number, not application-layer trust
- encrypted malicious traffic over `443` would require deeper inspection or anomaly detection
- the lab contains one IoT device rather than a large multi-device residential network
- lateral movement within a flat IoT subnet is not yet segmented at a per-device level
- the evaluation is strong at lab scale, but not yet validated at enterprise or ISP scale

These limitations should be stated explicitly because they motivate future work and help delimit the scope of the present contribution.

IX. Conclusion and Future Work

The proposed IoT Secure Gateway demonstrates that a gateway firewall can meaningfully reduce Mirai-style exposure by blocking WAN-to-IoT SSH/Telnet-style access, blocking insecure IoT-to-WAN propagation paths, and preserving legitimate HTTPS communication. The phase 3 evidence substantially strengthens this conclusion. The results show policy accuracy through direct rule-counter matches, efficiency through successful low-latency HTTPS runs and low observed resource usage, and scalability through burst testing that increased blocked traffic without disrupting legitimate communication.

An important contribution of this work is the role of the prior Mirai paper analysis in understanding the underlying cybersecurity problem. Studying the Mirai case in detail made it possible to move beyond a generic idea of "adding a firewall" and instead identify the specific failure points that make IoT botnet propagation effective: exposed management services, weak segmentation, and unrestricted outbound communication. That earlier analytical step was essential because it informed the topology, the forwarding rules, the attack simulation, and the evaluation criteria used throughout the implementation. In that sense, the literature analysis was not separate from the prototype; it was the foundation that made the prototype technically meaningful.

This solution is also relevant beyond the Docker laboratory environment. A similar architecture could be adapted to embedded security gateways built on lightweight edge devices, including low-power embedded computers and certain microcontroller-oriented platforms with networking capabilities. In such a deployment, a compact autonomous gateway could be configured to protect specific sections of a network in an organization, government office, school, clinic, or industrial dependency by enforcing segmentation and traffic restrictions without requiring each endpoint to support advanced host-based protection. This is especially important for IoT and operational devices that are resource-constrained, difficult to patch, or managed by non-specialist personnel.

Future versions of the system could also incorporate AI-assisted traffic filtering and anomaly response. Instead of relying only on static packet-filtering rules, the gateway could combine a secure default-deny baseline with AI-based or machine-learning-assisted traffic profiling to detect suspicious packet patterns, adapt filtering priorities, and react more quickly to emerging scanning or propagation behavior in near real time. Although such an extension would need careful control to avoid false positives and preserve explainability, it offers a promising direction for making IoT gateway protection more adaptive and operationally useful.

The project likewise has practical and commercial relevance. With stronger logging, centralized policy management, device profiling, and deployment automation, this research could evolve into a security package or managed service aimed at micro, small, and medium-sized enterprises that need affordable protection for cameras, sensors, access-control devices, and other connected systems. In that form, the solution could be delivered as a low-cost gateway appliance, a preconfigured security bundle, or a service offering tailored to organizations that do not have dedicated cybersecurity teams but still need meaningful network isolation and threat reduction.

Finally, the implementation highlights the relevance of the course in making this project possible. The work required combining concepts from network security, routing, firewall policy design, attack analysis, containerized deployment, evidence collection, and technical evaluation. The course therefore provided not only theoretical context, but also the practical structure needed to translate cybersecurity research into a functioning prototype. This connection between case-study analysis, hands-on implementation, and performance evaluation is one of the main strengths of the project and shows how academic work in cybersecurity can be developed into solutions with direct operational value.

Future work should focus on six main directions:

1) destination allowlisting and refinement of legitimate outbound communication policies

2) better logging, alerting, explainability, and forensic visibility
3) segmentation and policy customization for multiple IoT devices inside the same protected environment
4) feasibility analysis for lightweight embedded or microcontroller-based gateway deployments
5) AI-assisted packet inspection, anomaly scoring, and reactive rule adaptation in real time
6) larger-scale and more adversarial validation, including malicious encrypted traffic scenarios and more realistic enterprise deployments

Overall, this study demonstrates that network-level controls provide a practical and effective defensive layer for unmanaged or legacy IoT devices, that the Mirai case study offers a strong analytical foundation for implementation, and that the resulting approach has credible technical, educational, and commercial potential.

References

[1] M. Antonakakis *et al.*, "Understanding the Mirai Botnet," in *Proceedings of the 26th USENIX Security Symposium (USENIX Security '17)*, Vancouver, BC, Canada, Aug. 2017, pp. 1093–1110. [Online]. Available: https://www.usenix.org/system/files/conference/usenixsecurity17/sec17-antonakakis.pdf. Accessed: Mar. 2026.

[2] Cloudflare, "What is the Mirai Botnet?" [Online]. Available: https://www.cloudflare.com/learning/ddos/glossary/mirai-botnet/. Accessed: Mar. 2026.

[3] Cybersecurity and Infrastructure Security Agency (CISA), "Alert (TA16-288A): Heightened DDoS Threat Posed by Mirai and Other Botnets," Oct. 14, 2016. [Online]. Available: https://www.cisa.gov/news-events/alerts/2016/10/14/heightened-ddos-threat-posed-mirai-and-other-botnets. Accessed: Mar. 2026.

[4] Statista Research Department, "Average number of devices and connections per capita," 2023. [Online]. Available: https://www.statista.com/chart/32691/average-number-of-devices-and-connections-per-capita/. Accessed: Mar. 2026.